

Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Итоговый отчёт по модулю 1

Лабораторные работы №1 и №2
Логическое моделирование и подбор видеоигр
по курсу «Системы искусственного интеллекта»
Модуль 1. Базы знаний и онтологии

Выполнили:

Якименко Владислав, Р3313

Пахомов Игнат, Р3313

Проверил:

Жданов Андрей Дмитриевич

Содержание

1	Аннотация	1
2	Введение и анализ требований	1
2.1	Основные концепции и инструменты	1
3	ЛР 1. База знаний Prolog	3
3.1	Предметная область и словарь	3
3.2	Правила и порядок вычисления	3
3.3	Отсечение и полнота	4
3.4	Фактические ответы интерпретатора	4
4	ЛР 1. Онтология в Protégé	6
4.1	Перевод фактов и правил	6
4.2	Иерархия и свойства	7
4.3	Открытый мир и SWRL	7
4.4	Компетентностные запросы и проверка	7
5	ЛР 2. Система поддержки принятия решений	9
5.1	Постановка задачи и формат диалога	9
5.2	Архитектура и алгоритм	9
5.3	Сценарии использования	10
5.4	Ошибки и ограничения	11
6	Тестирование и отладка	12
6.1	Проверки Prolog	12
6.2	Проверки OWL и SWRL	12
6.3	Проверки диалога и DSS	13
6.4	Исправления по результатам аудита	13
7	Оценка и интерпретация результатов	14
8	Заключение	15

1 Аннотация

Разработана учебная система подбора видеоигр на основе логического представления знаний. Предметная область включает игровые режимы, сюжетность, сложность, независимость разработки, жанры и студии. База Prolog содержит 51 унарный факт, 15 бинарных фактов и семь правил. На её основе построена OWL-онтология с ограничениями, выводимыми классами и правилом SWRL. Консольная система поддержки принятия решений уточняет предпочтения и обращается к SWI-Prolog. Проверены положительные и отрицательные сценарии, а также соответствие результатов Prolog и Hermit. Основное ограничение решения связано с неполнотой учебной базы и различием предположений закрытого и открытого мира.

2 Введение и анализ требований

Цель модуля — освоить логическое моделирование, онтологии и проверяемый вывод, а затем использовать их в рекомендательном приложении. В отличие от модели, обученной на пользовательских оценках, здесь результат определяется явными фактами и правилами. Это позволяет объяснить рекомендацию и воспроизвести её при тех же данных. Один набор знаний используется в Prolog, в онтологии и в диалоговой программе.

Нумерация работ принята по выданному заданию «Системы ИИ»: лабораторная 1 включает базу знаний и её онтологическое представление, лабораторная 2 — DSS. В пособии ИТМО этим этапам соответствуют работы 1, 2 и 3 на страницах 56–58. Постановка задачи и структура расширенного отчёта также учитывают приложение 2 пособия и шаблон отчёта из задания.

Требуется не менее 20 унарных фактов, 10–15 бинарных фактов и 5–7 правил; нужны запросы разной сложности и комментарии. Онтология должна содержать классы, свойства, индивиды, иерархию, ограничения и демонстрацию вывода. Для DSS необходимы фиксированный формат входа, короткий диалог, обращение к знаниям, рекомендации с обоснованием и обработка некорректных данных.

База представляет учебный срез из 12 игр. Признаки «сложная» и «спокойная» являются принятыми условиями примера, а не результатами объективного измерения. Программа не оценивает возрастные рейтинги, актуальные цены и совместимость оборудования.

2.1 Основные концепции и инструменты

База знаний хранит не только сведения об объектах, но и правила получения новых утверждений. В Prolog предикат задаёт отношение, унификация связывает переменные с подходящими термами, а поиск с возвратом перебирает альтернативные доказательства. Такой механизм позволяет задавать условия подбора декларативно, без отдельного алгоритма для каждого сочетания предпочтений.

Онтология формализует общие понятия предметной области: классы, индивиды, свойства и аксиомы. OWL описывает логические ограничения, а reasoner проверяет их совместимость и выводит типы. Protégé служит редактором и средством просмотра, но не является базой данных, к которой обращается наше Python-приложение. SWRL дополняет модель правилом импликации; DL Query задаёт выражение класса, а SPARQL выбирает данные из RDF-графа.

SWI-Prolog исполняет правила и предоставляет библиотеки plunit для тестов и http/json для обмена данными. Python использует re для разбора строки, subprocess для запуска интерпретатора и unittest для проверки программы. RDFLib читает и сопоставляет графы OWL/Turtle, Owlready2 запускает HermiT. Последние две библиотеки нужны только для проверки онтологии, а не для диалога.

3 ЛР 1. База знаний Prolog

3.1 Предметная область и словарь

Объекты представлены атомами в `snake_case`: например, `portal_2`, `valve`. Переменные начинаются с заглавной буквы. Факт утверждает отношение, а правило задаёт условия его доказательства. Запятая означает конъюнкцию, точка с запятой — дизъюнкцию. Отрицание `\+ Goal` успешно, когда интерпретатор не смог доказать цель `Goal`.

Таблица 1: Факты исходной базы

Предикат	Число фактов
<code>game/1</code>	12
<code>single_player/1</code>	11
<code>multiplayer/1</code>	6
<code>story_driven/1</code>	7
<code>difficult/1</code>	5
<code>indie/1</code>	4
<code>relaxing/1</code>	3
<code>competitive/1</code>	2
<code>free_to_play/1</code>	1
Всего унарных	51
<code>developed_by/2</code>	10
<code>has_genre/2</code>	5

3.2 Правила и порядок вычисления

В файле `games.pl` факты сгруппированы по признакам и отношениям, далее записаны правила. Группы фактов и основные правила снабжены комментариями. Примеры запросов вынесены в `queries.md` и `run_examples.pl`, автоматические проверки — в `test_games.pl`; загрузка БЗ сама по себе не запускает демонстрации или тесты.

Семь правил описывают сюжетные одиночные игры, дружелюбные многопользовательские, напряжённые, сюжетные независимые, связь игры со студией, спокойные одиночные и подходящие новичку игры. Пример композиции условий:

```
recommended_for_beginner(Game) :-
    game(Game), single_player(Game),
    \+ difficult(Game), \+ competitive(Game).
```

Сначала выбирается конкретная игра, затем проверяются отрицания. Такая последовательность существенна: проверка отрицания со свободной переменной спрашивала бы об отсутствии любой сложной игры, а не о свойствах кандидата. В `intense_game/1`

игра Civilization VI удовлетворяет двум ветвям ИЛИ; `setof/3` устраняет повторения в итоговом наборе.

3.3 Отсечение и полнота

Предикат `!` отсекает альтернативы, возникшие до него в пределах текущего вызова. В запросе `story_driven(G), !, \+ developed_by(G,cd_projekt_red)` первым выбирается The Witcher 3. Отрицание не выполняется, а возвращаться к следующим сюжетным играм уже нельзя, поэтому ответ — `false`. Без отсечения находятся Portal 2, Hades, Hollow Knight, Celeste и Baldur's Gate 3. Здесь `cut` меняет множество решений и не подходит для получения всех рекомендаций.

3.4 Фактические ответы интерпретатора

Ниже приведены десять запросов, выполненных скриптом сбора результатов. `findall/3` собирает ответы в порядке поиска; `setof/3` возвращает отсортированный набор. Списки совпадают с отдельными интерактивными решениями.

Запрос 1

```
?- game(minecraft).  
true
```

Запрос 2

```
?- developed_by(portal_2,valve).  
true
```

Запрос 3

```
?- findall(G,developed_by(G,valve),L).  
[portal_2, counter_strike_2]
```

Запрос 4

```
?- findall(G,(single_player(G),story_driven(G)),L).  
[the_witcher_3, cyberpunk_2077, portal_2, hades, hollow_knight,  
celeste, baldurs_gate_3]
```

Запрос 5

```
?- findall(G,(game(G),(relaxing(G);free_to_play(G)),\+ competitive(G)),L).  
[minecraft, stardew_valley]
```

Запрос 6

```
?- findall(G,friendly_multiplayer(G),L).  
[portal_2, minecraft, stardew_valley, baldurs_gate_3]
```

Запрос 7

```
?- setof(G,intense_game(G),L).  
[celeste, civilization_vi, counter_strike_2, doom_eternal, hades,  
hollow_knight]
```

Запрос 8

```
?- findall(G,indie_gem(G),L).  
[hades, hollow_knight, celeste]
```

Запрос 9

```
?- findall(G,recommended_for_beginner(G),L).  
[the_witcher_3, cyberpunk_2077, portal_2, minecraft, stardew_valley,  
baldurs_gate_3]
```

Запрос 10

```
?- findall(G,relaxing_solo_game(G),L).  
[minecraft, stardew_valley, civilization_vi]
```

4 ЛР 1. Онтология в Protégé

4.1 Перевод фактов и правил

Онтология содержит 21 именованный класс, 24 именованных индивида, три объектных свойства, одно свойство данных, семь эквивалентностей и одно правило SWRL. Из 24 индивидов 12 являются играми, 8 — студиями и 4 — жанрами. Пространство имён: <http://example.org/itmo/games#>. OWL и Turtle представляют один RDF-граф из 329 троек; их изоморфность проверяется скриптом.

Таблица 2: Трассируемость Prolog и OWL

Prolog	OWL
game(G)	принадлежность VideoGame
single_player(G), multiplayer(G)	SinglePlayerGame, MultiplayerGame
story_driven(G), difficult(G)	StoryDrivenGame, DifficultGame
indie(G), relaxing(G)	IndieGame, RelaxingGame
competitive(G), free_to_play(G)	CompetitiveGame, FreeToPlayGame
developed_by(G,S)	developedBy(G,S), обратное developedGame(S,G)
has_genre(G,T)	hasGenre(G,T)
solo_story_game(G)	SoloStoryGame: SinglePlayerGame and StoryDrivenGame
friendly_multiplayer(G)	MultiplayerGame and NonCompetitiveGame
intense_game(G)	DifficultGame or CompetitiveGame
indie_gem(G)	IndieGame and StoryDrivenGame
relaxing_solo_game(G)	RelaxingGame and SinglePlayerGame
recommended_for_beginner(G)	SinglePlayerGame and NonDifficultGame and NonCompetitiveGame
studio_game(G,S)	StudioGame — проекция на G; S извлекается через developedBy

Двухаргументный предикат нельзя полностью заменить одним классом: StudioGame показывает наличие разработчика, но для получения пары игра–студия необходимо объектное свойство. В исходном наборе десять явно известных таких пар. По семи остальным унарным результатам, включая отдельную проверку SWRL, множества экземпляров сопоставляются с результатами Prolog автоматически.

4.2 Иерархия и свойства

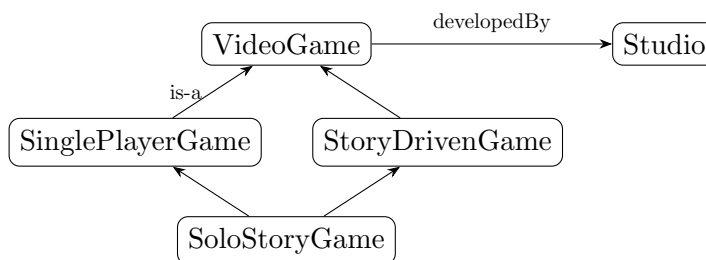


Рис. 1: Фрагмент иерархии и связи между играми и студиями

Классы VideoGame, Studio и Genre попарно несовместимы. При этом SinglePlayerGame и MultiplayerGame совместимы: Portal 2 относится к обоим. Domain свойства developedBy равен VideoGame, range — Studio; у developedGame направления обратные. У hasGenre domain VideoGame, range Genre. Domain и range служат аксиомами вывода типов, а не проверками ввода в стиле реляционной базы.

Добавлен DataProperty title с диапазоном xsd:string. У VideoGame задано title exactly 1 xsd:string, у StudioGame — developedBy min 1 Studio. Названия являются дополнительными данными отображения относительно исходного Prolog. Минимальная кардинальность в OWL допускает неизвестный объект; отсутствие явно записанного значения само по себе не делает онтологию противоречивой. В отличие от этого, два разных строковых title нарушают exactly 1.

4.3 Открытый мир и SWRL

Prolog трактует отсутствие доказательства как неудачу цели. OWL не выводит отрицание из отсутствия факта. Поэтому для данного конечного набора явно заданы NonCompetitiveGame и NonDifficultGame, несовместимые с положительными категориями. Эти утверждения добавлены при переводе и требуют пересмотра при изменении набора фактов.

Для отдельной демонстрации SWRL задано правило:

```
SinglePlayerGame(?g) ^ StoryDrivenGame(?g)
-> RuleVerifiedStoryGame(?g)
```

Этот класс не имеет Equivalent To, поэтому его семь экземпляров демонстрируют именно срабатывание правила. HermiT проверил правило для именованных индивидов. Остальные логические определения выражены штатными конструкциями OWL.

4.4 Компетентностные запросы и проверка

В Protégé файл открывается через File → Open. После запуска Reasoner → HermiT → Start reasoner на вкладке DL Query выбирается Instances и вводятся имена ниже. При отображении prefixed names используется ведущий символ двоеточия.

1. `SoloStoryGame`: `baldurs_gate_3`, `celeste`, `cyberpunk_2077`, `hades`, `hollow_knight`, `portal_2`, `the_witcher_3`.
2. `FriendlyMultiplayerGame`: `baldurs_gate_3`, `minecraft`, `portal_2`, `stardew_valley`.
3. `IntenseGame`: `celeste`, `civilization_vi`, `counter_strike_2`, `doom_eternal`, `hades`, `hollow_knight`.
4. `IndieGem`: `celeste`, `hades`, `hollow_knight`.
5. `BeginnerRecommendedGame`: `baldurs_gate_3`, `cyberpunk_2077`, `minecraft`, `portal_2`, `stardew_valley`, `the_witcher_3`.
6. `RuleVerifiedStoryGame`: тот же набор из семи игр, что у `SoloStoryGame`.

Наборы выше получены `HermiT` в автоматическом запуске, а не переписаны из предполагаемого результата. Файл `verification.json` фиксирует вывод. Для визуального просмотра можно включить `OntoGraf` и раскрыть `VideoGame`; рисунок в отчёте показывает фрагмент тех же отношений. Дополнительно SPARQL с `FILTER NOT EXISTS` выбирает сюжетные игры без записанной связи с `Valve`: `baldurs_gate_3`, `celeste`, `cyberpunk_2077`, `hades`, `hollow_knight`, `the_witcher_3`. Это проверка отсутствия тройки в графе, а не OWL-доказательство отрицания. Запрос `DL not (developedBy value valve)` в общем случае не эквивалентен такому фильтру.

5 ЛР 2. Система поддержки принятия решений

5.1 Постановка задачи и формат диалога

Система рекомендует игры на основе предпочтений и двух уточнений. Ввод задаётся списком предпочтений через запятую: «RPG, инди». Для совместимости принимается и форма «Мне нравятся: RPG, инди»: вводная фраза необязательна. Допустим словарь известных значений, а не произвольное описание на естественном языке. Возраст не включён: исходная база не содержит рейтингов, и приложение не может обосновать возрастную фильтрацию. Пример возраста в задании является иллюстрацией возможных слотов, а не единственно допустимым форматом.

Словарь нормализации объединяет русские и английские названия в пять канонических предпочтений: ролевые игры, инди, головоломки, шутеры и пошаговые стратегии. Например, RPG, РПГ и «ролевая игра» дают один атом `role_playing_game`. Словарь полностью приведён в README приложения. Инди является производственным признаком. Для него используется предикат `indie/1`; жанры проверяются отношением `has_genre/2`.

Первый уточняющий вопрос: режим «один», «вместе» или «любой». Второй: нужна ли игра для новичка («да» или «нет»). Пустой и неизвестный ответ приводит к повтору соответствующего вопроса. Рекомендации выводятся только после завершения диалога.

5.2 Архитектура и алгоритм

Python отвечает за диалог, проверку формата и отображение. Модуль `bridge.pl` загружает `games.pl` первой лабораторной и выполняет логический поиск. Входные параметры передаются JSON через стандартный ввод отдельного процесса SWI-Prolog, результаты возвращаются через стандартный вывод. Командная оболочка не используется; пользовательский текст не подставляется в код цели.

```
INPUT preference line
REMOVE optional prefix; SPLIT comma-separated preferences
NORMALIZE known aliases; REMOVE duplicates
ASK mode; ASK beginner filter
FOR each game in the knowledge base:
    REQUIRE selected mode
    REQUIRE beginner rule if requested
    COLLECT matching preferences
    KEEP game if at least one preference matched
SORT by match count descending, then game identifier
OUTPUT recommendations and matched conditions
```

Предпочтения объединяются по ИЛИ, а уточнения применяются по И. Оценка

кандидата g равна числу совпавших различных предпочтений:

$$s(g) = |\{t \in T : \text{matches}(g, t)\}|.$$

Требуется $s(g) > 0$. Одинаковые оценки упорядочиваются по идентификатору, что делает результат воспроизводимым. В текущих данных комбинации жанров и инди не пересекаются, поэтому фактические оценки в сценариях равны единице. Счёт поддерживает расширение базы, но не изображает вероятности или качество игр.

```
candidate(Tags, Mode, Experience, G, Score, Matched) :-
    game(G), mode_fits(G, Mode),
    experience_fits(G, Experience),
    findall(T, (member(T, Tags), matches(G, T)), Raw),
    sort(Raw, Matched), length(Matched, Score), Score > 0.
```

При N играх и K предпочтениях логическая фильтрация выполняет порядка NK проверок совпадения; сортировка R результатов требует порядка $R \log R$ сравнений. Для 12 игр стоимость запуска интерпретатора важнее самого поиска. Индексацию простых фактов обеспечивает SWI-Prolog; отсечение не требуется.

5.3 Сценарии использования

Каждая строка ниже соответствует реальному вызову моста Prolog, записанному в evidence.json. Начало входной строки во всех случаях — «Мне нравятся:».

Таблица 3: Входы, рекомендации и объяснения

Предпочтения	Уточнения	Результат и основание
RPG	один; новичок	cyberpunk_2077, the_witcher_3: жанр RPG, single_player и recommended_for_beginner.
Инди	вместе; новичок	stardew_valley: indie, multiplayer и правило новичка.
Головоломка	вместе; любой опыт	portal_2: has_genre(puzzle) и multiplayer.
Шутер	один; любой опыт	doom_eternal: has_genre(first_person_shooter) и single_player.
Стратегия	любой режим; любой опыт	civilization_vi: has_genre(turn_based_strategy).
RPG	вместе; любой опыт	Пустой список: ни у одной игры с записанным RPG нет multiplayer.
Шутер	один; новичок	Пустой список: Doom Eternal отмечена difficult и исключена правилом новичка.

Предпочтения	Уточнения	Результат и основание
RPG, инди	один; новичок	cyberpunk_2077, stardew_valley, the_witcher_3: каждый совпал с одним предпочтением.

Первые пять сценариев соответствуют следующим логическим запросам (G — игра, S — балл, M — совпавшие предпочтения):

```
candidate([role_playing_game],solo,beginner,G,S,M).
candidate([indie],multi,beginner,G,S,M).
candidate([puzzle],multi,any,G,S,M).
candidate([first_person_shooter],solo,any,G,S,M).
candidate([turn_based_strategy],any,any,G,S,M).
```

Для Stardew Valley проверяются признаки indie и multiplayer, затем правило recommended_for_beginner. Итоговый балл равен 1, а список объясняющих совпадений равен [indie].

5.4 Ошибки и ограничения

Неизвестное предпочтение «гонки», пустая строка и завершающая запятая отвергаются до запуска Prolog. Повторение RPG и РПГ не увеличивает балл. При конфликте режима и признаков система сообщает, что совпадений в БЗ нет, и выводит «ничего не нашлось». Пользователь может повторить запуск с другими условиями. Отсутствие swipl приводит к сообщению «swipl не найден»; на выполнение запроса установлен таймаут 15 секунд. EOF и Ctrl+C завершают диалог.

Пустой список не доказывает отсутствие подходящей игры в мире. В частности, жанр Baldur's Gate 3 в этой БЗ не задан, поэтому система не рекомендует её по RPG. Такой результат согласован с данными и явно документирован. Признак «подходит новичку» не является рекомендацией по возрасту.

В консольном объяснении выводятся совпавшие теги, а затем короткие обозначения solo или multi для выбранного режима и «для новичка» при включённом фильтре. Например, строка portal_2 (1): совпало: puzzle; solo означает одно совпавшее предпочтение и наличие одиночного режима. Параметры запроса показываются отдельной JSON-строкой; исходящие поля — tags, mode, experience, поля объектов ответа Prolog — game, score, matched.

6 Тестирование и отладка

6.1 Проверки Prolog

В SWI-Prolog 10.0.2 выполнены 13 тестов plunit. Проверены положительный факт, отсутствующая игра, игры Valve, сюжетные одиночные игры, сочетание И/ИЛИ/НЕ, дружелюбный мультиплеер, множество напряжённых игр, сюжетные инди, правило новичка, спокойные одиночные игры и десять пар игра–студия. Ещё два теста демонстрируют false с отсечением и пять ответов без него. Все 13 тестов прошли. Отдельный запуск десяти демонстрационных запросов сохранён в evidence.json и использован для листингов отчёта.

6.2 Проверки OWL и SWRL

Использованы RDFLib 7.6.0, Owlready2 0.51 и встроенный в Owlready2 HermiT. Это автоматический запуск того же семейства reasoner’ов, которое доступно в Protégé; он не подменён простым обходом RDF-троек. Java взята из установленного Protégé. После классификации наборы экземпляров шести определяемых классов и SWRL-класса сравнены с ответами соответствующих предикатов Prolog.

Таблица 4: Проверки онтологии и фактический результат

Проверка	Результат
Изоморфность Turtle и RDF/XML	Графы совпадают
Консистентность и выполнимость именованных классов	Противоречий нет
Семь наборов выведенных типов против Prolog	Все совпали
Обратная связь developedGame для Valve	portal_2, counter_strike_2
SoloStoryGame как подкласс SinglePlayerGame	Вывод подтверждён
SPARQL для сюжета без записанной связи с Valve	6 игр
Minecraft одновременно Competitive и NonCompetitive	Обнаружена неконсистентность
Число 42 вместо строкового title	Обнаружена неконсистентность
Два различных title при exactly 1	Обнаружена неконсистентность

Негативные изменения вносятся в отдельные миры Owlready2 в памяти. Файл исходной онтологии ими не изменяется. Это даёт возможность отличить проверку корректного примера от проверки способности ограничений обнаруживать ошибки.

6.3 Проверки диалога и DSS

В Python unittest выполнены семь тестовых методов с параметризованными случаями. Восемь интеграционных сценариев вызывают реальный SWI-Prolog. Дополнительно проверены шесть неверных входных строк, четыре неверных набора аргументов программного интерфейса, обе формы ввода (с префиксом и без него), синонимы и дубликаты, отсутствие swipl, повтор вопросов после ошибки и корректное завершение по EOF. Все тестовые методы прошли; внешних сетевых сервисов приложение не требует.

```
swipl -q -s lab_1/test_games.pl -g run_tests -t halt
python3 -m unittest discover -s lab_2 -v
python3 lab_1/part_2_ontology/verify_ontology.py
python3 reports/collect_evidence.py
```

Команды выполняются из module_1. Для третьей команды нужны зависимости из requirements-checks.txt и Java; для DSS достаточно стандартной библиотеки Python и установленного SWI-Prolog. Пути к БЗ вычисляются относительно файлов программы, поэтому запуск не зависит от текущей рабочей папки.

6.4 Исправления по результатам аудита

Исходные факты и семь правил первой работы соответствовали количественным требованиям. Дополнены автоматические проверки, свойство данных title, кардинальности, явный третий уровень иерархии и отдельно тестируемое SWRL-правило. Уточнены документация по открытому миру и трактовка StudioGame. Дубликат при дизъюнкции устранён на уровне выборки, а не изменением фактов. Перед публикацией проверена обновлённая версия трёх файлов программы: games.pl, bridge.pl и recommender.py. Описание необязательного префикса ввода и коротких консольных сообщений приведено в соответствие с кодом; тест ошибочного ввода RPG заменён на неизвестный RGB, поскольку RPG без префикса теперь является допустимым вводом.

7 Оценка и интерпретация результатов

Prolog удобен для последовательности вычислений, отрицания как неудачи и формирования списков кандидатов. OWL удобен для описания общей модели, наследования, ограничений и проверки непротиворечивости. Эквивалентные классы позволяют классифицировать индивиды без отдельного запуска каждого правила. При этом одинаковое внешнее слово «не» имеет различную семантику: в Prolog достаточно отсутствия доказательства, а в OWL требуется логическое основание отрицания.

Приложение использует Prolog как источник решений, поскольку условия диалога естественно выражаются предикатами. Онтология выполняет роль формальной модели и независимой проверки выбранных классификаций. Совпадение результатов подтверждено для заданного конечного набора; оно не доказывает эквивалентность двух формализмов для произвольного расширения базы.

Основные количественные требования выполнены: 51 унарный и 15 бинарных фактов, семь правил, десять демонстрационных запросов Prolog и шесть компетентностных запросов к онтологии. Иерархия, типизированные свойства, кардинальности и SWRL проверены reasoner'ом. DSS использует ввод по заданному формату с необязательным префиксом, нормализацию, два уточнения, ранжирование и объяснение; восемь сценариев покрывают как выдачу рекомендаций, так и пустую выборку.

Полнота рекомендаций ограничена пятью записанными жанровыми связями. Следующий этап развития — расширение фактов с сохранением их происхождения, пересмотр явных отрицательных категорий и добавление подтверждённых возрастных ограничений. Для большого каталога можно использовать постоянный процесс интерпретатора и отдельное хранилище знаний. До расширения данных такие изменения не улучшают качество рекомендации сами по себе.

8 Заключение

Выполнен переход от фактов о видеоиграх к правилам Prolog, онтологии OWL и диалоговой системе поддержки принятия решений. Количество фактов и правил соответствует заданию, запросы демонстрируют переменные, логические операции и вывод. Онтология задаёт иерархию, домены и диапазоны, несовместимость классов, свойство данных, кардинальности и SWRL-правило.

Тестирование показало совпадение выбранных классификаций Prolog и HermiT, обнаружение искусственно внесённых противоречий и корректную обработку положительных и отрицательных сценариев диалога. Пользователь получает идентификаторы рекомендованных игр и основания их отбора. Практическая польза подхода состоит в прозрачности: изменение результата можно проследить до факта, правила или введённого ограничения.

Решение применимо к небольшому каталогу, где правила выбора известны и требуется объяснять каждое решение. Аналогичный подход можно использовать для подбора учебных материалов или программных инструментов, заменив словарь и факты соответствующей предметной области. Повторное использование знаний обеспечивается тем, что диалог не содержит собственного списка игр и загружает базу первой лабораторной.

Ограничения определены явно: жанры заполнены частично, сложность принята как учебный признак, а отрицательные категории OWL заданы для конечного набора. Поэтому успешные тесты подтверждают корректность реализации относительно этой базы, но не полноту сведений о реальных играх. При расширении каталога необходимо дополнять факты, актуализировать отрицательные категории и повторять тесты согласованности. Цель модуля достигнута: создан полный воспроизводимый цикл от логического описания знаний до объяснимой рекомендации.

Использование ИИ-инструментов

При подготовке работы использовался ИИ-ассистент OpenAI Codex (ChatGPT) при подготовке проверок и оформлении отчёта. Полученные решения проверялись запуском программ и автоматических тестов; для онтологии применялся логический вывод HermiT. Результаты этих проверок приведены в отчёте.